Computing and Visualizing the Visibility Polygon

Tijana Jakšić

September 14, 2024

Contents

1 Introduction **1**

2 Problem Definition **1**

3 Implementation **2**
3.1 Data Structures 2
3.1.1 Container Data Structures 3
3.2 Algorithm 4

4 Complexity Analysis **5**
4.1 Time Complexity 5

5 Space Complexity **5**

1 Introduction

Given a top-down map of the plane with obstacles in the form of segments, and a viewpoint p, the visibility polygon is the subset of the plane consisting of all points visible from p.

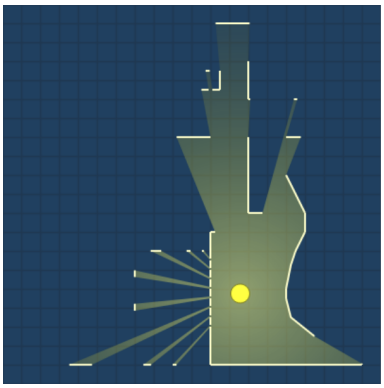


Figure 1: Illustration of the region visible from a given point [2]

The visibility polygon is related to the well-known art gallery problem, and it has practical applications in video games, robotics and navigation.

The first paper to describe computing the visibility polygon using the rotational sweep-ray technique is [1]. This project implements a similar algorithm in C++, relying on the SFML library for visualization. An interactive visualization is implemented in which the user sets the coordinates of the viewpoint with the mouse position. In addition, the coordinates of the obstacles can be specified by editing the file `segments.txt` before running the program.

2 Problem Definition

Mathematically, the visibility polygon can be defined as follows: Let a set of segments S (*obstacles*) be given in the plane $\mathbf{R}^2$, together with a point p that does not lie on any of the obstacles. The visibility polygon V is then the set of points q in $\mathbf{R}^2$ such that the segment pq does not intersect any obstacle.

Note that, defined this way, the visibility polygon may be an unbounded subset of the plane. The first additional assumption made for the implementation is that the set S is finite. We further assume that we are not working with the whole plane, but that the set of obstacles contains segments forming a closed region in which all other segments and the viewpoint lie. This is acceptable because a boundary can always be added afterwards to any finite set of segments.

Furthermore, it is assumed that the interiors of the given segments do not intersect. If the given segments intersect at interior points, the input set of

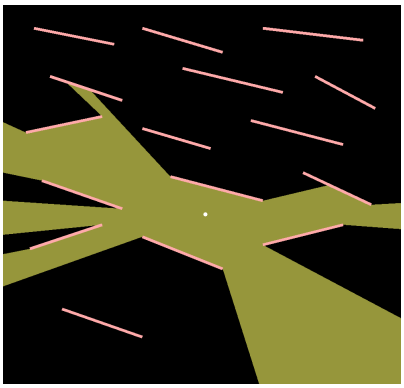


Figure 2: Visibility polygon

segments would have to be preprocessed, but this is not implemented in this project.

Therefore, the algorithm takes as input a set of obstacles and a viewpoint satisfying the following assumptions:

- The obstacles are segments whose interiors do not intersect
- There is a subset of segments forming a closed region that contains all other segments and the viewpoint

As output, the algorithm returns a vector of points describing the visibility polygon.

3 Implementation

3.1 Data Structures

Auxiliary geometric data structures:

Structure: Point

x: float
y: float

Structure: Segment

a: Point
b: Point

Data structure describing an event:

Structure: Event

point: Point
segment: Segment
end: bool

The attribute *end* stores whether *point* is the start or the end vertex of the segment. This can be determined from the orientation of the triangle $(p, segment.a, segment.b)$.

3.1.1 Container Data Structures

Event queue. Events are stored in a vector $\mathcal{E}$. Since the sweep-ray technique requires the events to be sorted, an event comparator CompareEvents is introduced. The main criterion is the angle of the vector $p \to e.point$. If several events lie on the same viewing ray, this implementation treats more distant events as smaller; then, if both the angle and the distance are identical, it treats *end* events as smaller.

Status. The status is a data structure $\mathcal{T}$ that should store the sequence of segments intersected by the rotating sweep ray, correctly ordered so that $\min \mathcal{T}$ is the first visible segment. In addition, $\mathcal{T}$ should support efficient search, insertion and deletion of elements. Therefore, an ordered balanced binary tree is used. A comparator CompareSegments is introduced, describing the "in front of" relation. The relation depends on the viewpoint p. Note that the in-front-of relation is not the same as the relation of distance from p.



Figure 3: Two segments for which the in-front-of relation does not coincide with distance from p

Segment s_1 is in front of segment s_2 if there exists a viewing ray r from the viewpoint p that intersects both segments and, denoting these intersections by I_1 and I_2, we have $d(I_1, p) < d(I_2, p)$. In other words, if its distance to the viewpoint along the viewing ray is smaller. This is precisely why it is important

that the interiors of the given segments do not intersect—so that this relation is well defined.

3.2 Algorithm

Below is the main algorithm, which takes as input the viewpoint p and the set of obstacle segments $\mathcal{S}$, and returns as output the vector of vertices of the visibility polygon $\mathcal{V}$.

Algorithm: VisibilityPolygon($p, \mathcal{S}$)

```
1:  Initialize an empty event vector E
2:  for each segment s in S do
3:      For each vertex of segment s, add the corresponding event to E
4:  Sort E according to CompareEvents
5:  Initialize an empty status structure T
6:  Initialize an empty vector V
7:  for each e in E do
8:      ProcessEvent(e)
```

Next is the event-processing algorithm, which takes an event e as input and incrementally updates $\mathcal{V}$ accordingly. The viewing angle is the angle of the vector $p \to e.point$, and the viewing ray is the ray starting at p with the viewing angle.

Algorithm: ProcessEvent(e)

```
1:  Update the viewing angle
2:  if e.end = true then
3:      Delete e.segment from T
4:      if e.segment = min(T) then     {the segment was visible}
5:          Add e.point to V
6:          if T ≠ ∅ then
7:              Add to V the intersection of the new min(T) with the view-
                ing ray
8:  else   {e.end = false}
9:      if T ≠ ∅ then
10:         Find the intersection point q of min(T) with the viewing ray
11:     Add e.segment to T
12:     if e.segment = min(T) then     {new closest segment}
13:         if point q was found then
14:             Add q to V
15:         Add e.point to V
```

4 Complexity Analysis

4.1 Time Complexity

Let n be the number of given obstacle segments. The main algorithm VisibilityPolygon reads the events in $O(n)$ time, sorts them in $O(n \log n)$ time, and then applies ProcessEvent $O(n)$ times. The only operations in ProcessEvent that do not run in linear time are updating and searching the status $\mathcal{T}$, which run in logarithmic time. Since $\mathcal{T}$ can contain at most n elements, the algorithm VisibilityPolygon runs in $O(n \log n)$ time.

5 Space Complexity

The additional data structures used are the event queue, which can contain at most $2n$ elements (one event per segment vertex), and the balanced ordered binary tree $\mathcal{T}$, which stores segments and can therefore contain at most n elements.

This implementation stores the result in a vector before drawing, but this is not necessary: the triangular pieces of the result could be drawn one at a time. This choice does not affect the asymptotic space complexity, since the result cannot have more than $4n$ elements (there are $2n$ events and each event adds at most 2 points).

Therefore, the space complexity of this algorithm is linear.

References

[1] Tetsuo Asano. An efficient algorithm for finding the visibility polygon for a polygonal region with holes. *Institute of Electronics, Information, and Communication Engineers*, 68:557–559, 1985.

[2] Red Blob Games. 2d visibility. `https://www.redblobgames.com/articles/visibility/`, 2012. Accessed: 14.9.2026.